

HBnB Part 1 - Technical Documentation

Blueprint for Holberton AirBnB clone. An introduction to UML, software architecture and visualising object-oriented programming

Anthony Joy, Zac Malkoun, Lachlan McKenna, Sam Thompson

Introduction

This section of the HBnB project is for creating comprehensive technical documentation that will serve as the foundation for the development of the AirBnB clone. This documentation will serve as the basis for the understanding of the overall architecture, the design of the business logic, and the interactions within the systems.

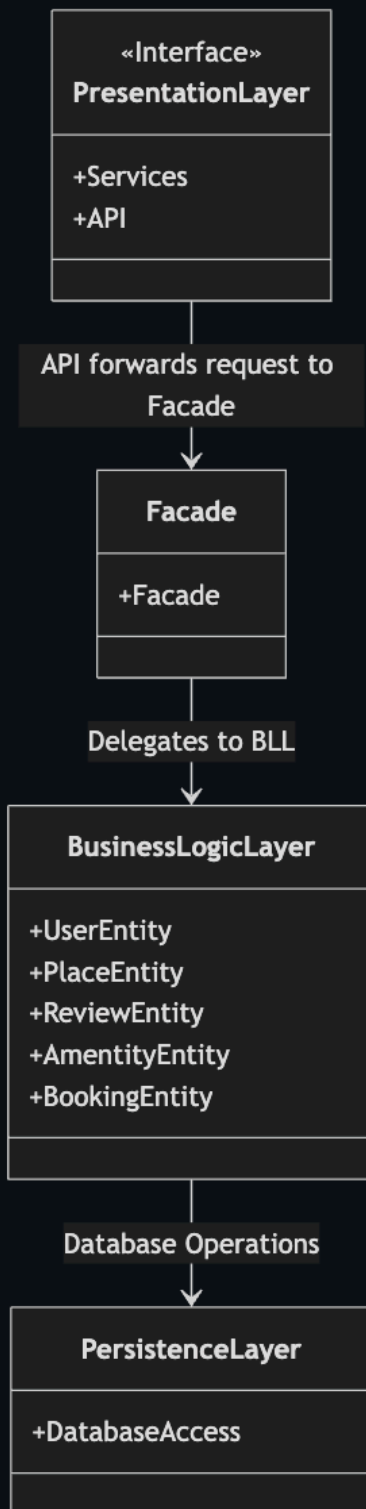
We have been tasked with creating a simple AirBnB clone which will allow users to perform these main operations:

- **User Management** - Users will be able to register or update their profiles, and be identified as consumers, administrators or BnB Hosts.
- **Place Management** - Users can list properties they own, giving details including its name, a description of the property, price, and its location (using latitude and longitude). These properties will also have its amenities listed.
- **Review Management** - Reviews can be left on properties the users have visited, with a rating and a comment
- **Amenity Management** - The application will manage amenities that can be associated with places

The application will follow a layered architecture divided into the presentation layer, business logic layer, and the persistence layer. These will be explained in detail further into the document.

High-Level Architecture

This section is for the high-level package diagram, illustrating the three-layer architecture of the HBnB project, with the communication between these layers via the facade.



Working from the top down, we have

Presentation Layer:

- This layer contains the API, and is the only layer which the user will interact with
- This layer sends requests to the business logic layer via the facade, which delegates the tasks to the return the correct response to the user
- The presentation layer is unaware of the persistence layers existence.

Business Logic Layer:

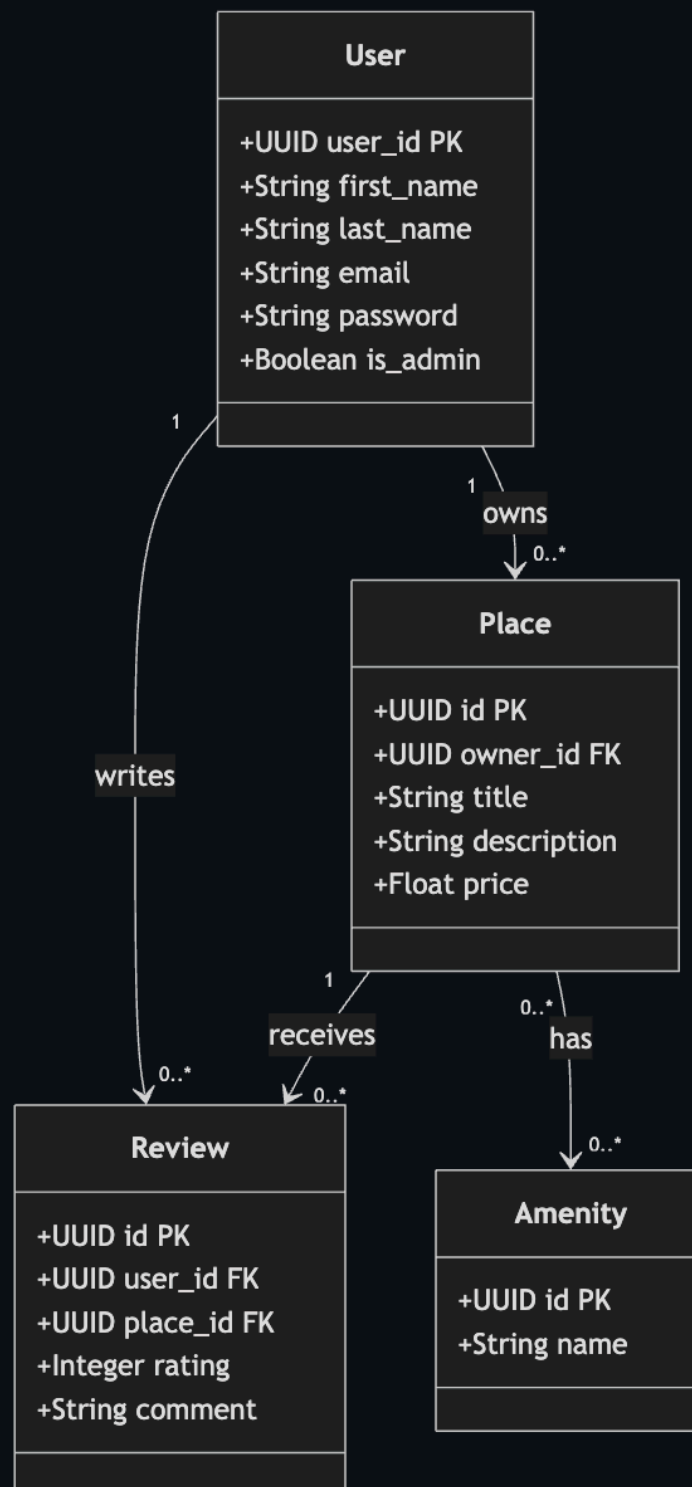
- The BLL holds the main entities required for use and will hold the logic and connections between the entities (explained in the class diagram of the BLL below)
- There is communication between both layers though it only sends requests to the persistence layer if the logic is passed

Persistence Layer:

- Handles the connection to the database
- Deals with the reading and writing of data such as changing values of a property or registering new users
- This layer does not know of any logic from the BLL and only does the task given to it by the above layer

Facade Patter - Detailed Class Diagram for the BLL

The diagram below shows the entities within the business logic layer, their attributes, methods, and the relationships between the entities, focusing on the user, place, review, and amenity entities.



User Entity:

- Represents the users of the application
- Each user contains personal information such as their first and last name, email address, and a password
- The `is_admin` attribute is a boolean and identifies administrators
- Each user is also assigned a primary key to uniquely identify them

Place Entity:

- Represents properties listed by users
- Each property includes a title, description, price, and latitude and longitude
- A place belongs to a single user and can contain multiple amenities
- Methods are included for creating, updating, listing and deleting properties

Review Entity:

- Represents feedback left by users on places where they've stayed
- Contains ratings and comments left by users
- Each review is associated to one user and one place

Amenities Entity:

- Represents features that can be associated with places
- Can be linked with multiple places

These entities have relationships between each other as well

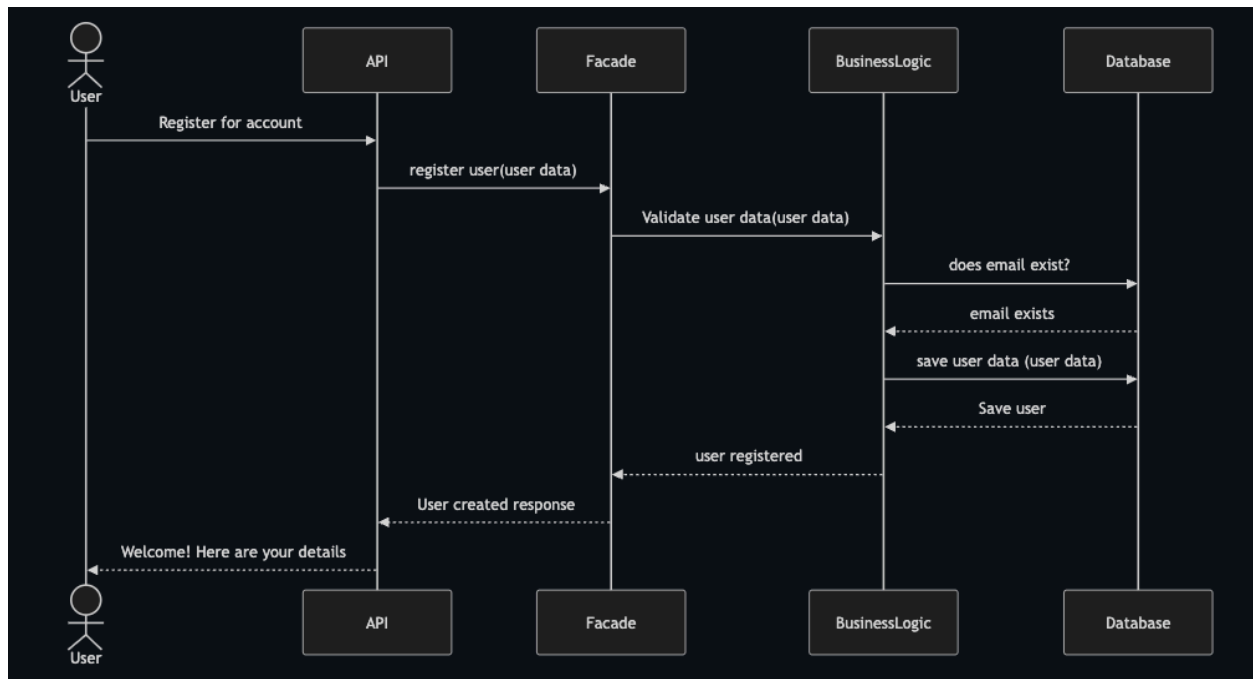
- User and Place have a one to many relationship, where one user can own multiple places, but each place can only belong to one user
- User and Review have a one to many relationship, users can write many reviews but each review belongs to one person
- Place and Review have a one to many relationship, a place can have many reviews but each review is associated with a single place
- Place and Amenity have a many to many relationship, places can have many amenities and the amenities can exist between many places

Each entity of the business layer also has a UUID4 identifier to make sure each attribute is unique.

Sequence Diagrams for API Calls

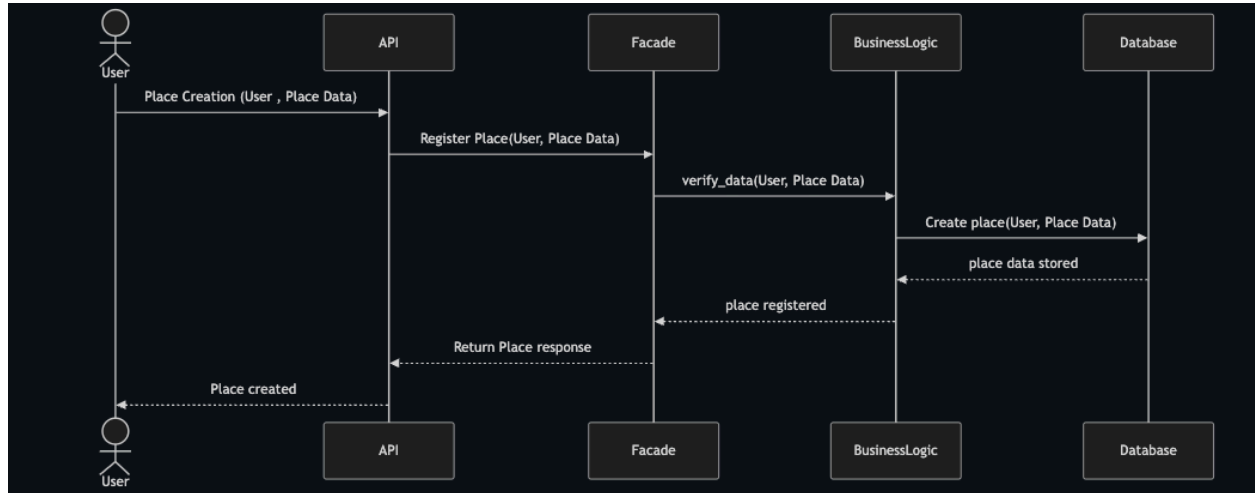
This section showcases some specific examples of different API calls a user to try, displaying how the presentation, business logic and persistence layers communicate with each other.

User Creation:



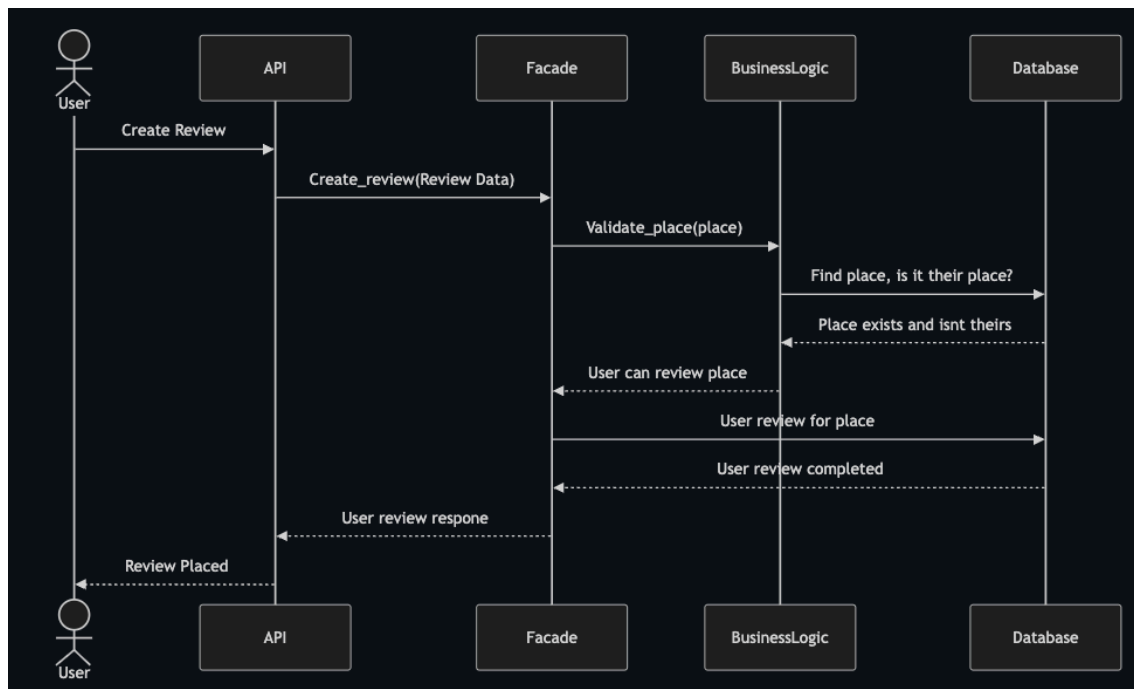
The API receives the registration request and passes it onto the facade. This sends it towards the BLL for validation, checking whether the email is unique and doesn't exist within the database. The user will be created if the email is unique, or will request a new email from the user if already used.

Place Creation:



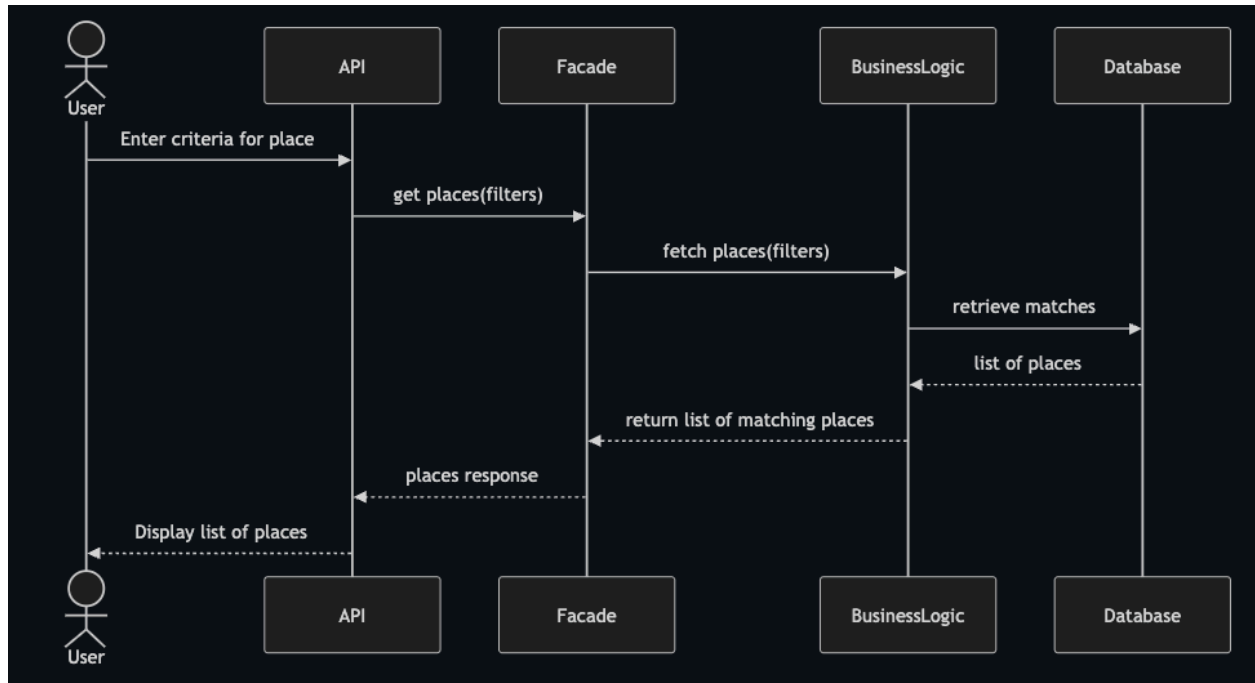
This follows the same sort of pattern, user will request to register a property and the API passes on to the facade. This sends it toward the business logic layer to verify user identity and place data which is sent and stored in the database and in return the listing is created.

User Review:



The user creates a review and the API receives the review request. Is then checked if the user owns the place in the BLL. If the user isnt the owner, it can be reviewed and a review is made directly in the facade, otherwise the review is not accepted.

Fetching a list of places:



This is just a filter with no checks required. It sends a request of places through the business logic layer into the database which returns matching places according to the filters (such as price range, location range, or listed amenities) which will be displayed to the user.